

LearnHub MERN E-Learning Platform

Comprehensive Technical Interview & Code Verification Guide | 50 Core Questions + 10 DSA Java Solutions

Author / Candidate: Fresher MERN Stack Developer	Target Role: Full-Stack Web Developer
Tech Stack: MongoDB, Express.js, React.js, Node.js	Authentication: JWT & Bcryptjs Role Control
Repository Scope: Grounded 100% in local project codebase	Code Audit Status: Clean (Zero Comments)

Document Purpose: This guide prepares you to confidently answer interview questions about your newly created **LearnHub** project. Every answer and code reference is strictly matched to your exact codebase (e.g., `authController.js`, `courseController.js`, `enrollmentController.js`, `AuthContext.jsx`, `LessonView.jsx`). An interviewer asking these questions will immediately verify that you wrote every line of code yourself.

Part 1: 50 Project & Stack Interview Questions

1. Project Overview & Problem Statement

Q1: What is LearnHub and what specific problem does it solve?

■ **Relevant File:** Project Root (`server/` & `client/`)

Answer: LearnHub is a full-stack MERN e-learning web application that connects instructors (Admins) with learners (Students). It solves the problem of simple course delivery by enabling admins to create and manage structured course syllabi (lessons with text or embedded videos) and allowing students to enroll in courses, study lessons sequentially, and visually track their completion progress through a dynamic percentage bar.

Interviewer Follow-up Questions:

- *Follow-up 1: How does LearnHub separate Admin privileges from Student privileges?*
- *Follow-up 2: Why simulate enrollment instead of adding complex payment processors?*

Deep Dive Answer: Admin privileges allow full **CRUD** operations on courses and lesson syllabi along with inspecting student enrollment progress lists. Students can only browse, enroll, view lesson content, and mark their own lessons as complete. Payment processing was simulated to keep the application lightweight, responsive, and focused on core LMS functionality.

Q2: What are the main user roles in LearnHub and how are they assigned during registration?

■ **Relevant File:** `server/controllers/authController.js (registerUser)`

Answer: LearnHub supports two user roles: 'admin' and 'student'. Role assignment occurs during signup inside `registerUser`. The registration payload accepts an optional `adminCode`. If `adminCode` matches `process.env.ADMIN_SECRET_CODE` exactly, the role is assigned as 'admin'. If left blank or incorrect, the user is automatically registered as a 'student'.

Interviewer Follow-up Questions:

- Follow-up 1: Is `ADMIN_SECRET_CODE` ever exposed in API responses or JWT tokens?
- Follow-up 2: What happens if an unauthorized user passes role: 'admin' directly in the request body?

Deep Dive Answer: `ADMIN_SECRET_CODE` is kept strictly on the server in `.env` and is never returned in any response. Attempting to send role: 'admin' in the JSON body is ignored because the role is determined strictly server-side based on the secret code check.

2. Why MERN Stack?

Q3: Why did you choose the MERN stack (MongoDB, Express, React, Node) over SQL/Spring?

■ **Relevant File:** `System Architecture`

Answer: The MERN stack allowed end-to-end JavaScript development, enabling code reuse and seamless JSON data transmission between frontend React components and backend Express APIs. MongoDB's document model fits course syllabi natively because lessons can be referenced or populated as nested arrays inside course objects without requiring complex SQL joins.

Interviewer Follow-up Questions:

- Follow-up 1: How does Node's non-blocking I/O model benefit an e-learning platform?
- Follow-up 2: When would a relational database like PostgreSQL be better than MongoDB for LearnHub?

Deep Dive Answer: Node's asynchronous event loop efficiently handles concurrent student API requests (such as lesson progress updates and course browsing) without spawning thread-per-request overhead. PostgreSQL would be preferred if complex ACID transactions involving financial billing or multi-tier multi-tenant enterprise organizations were required.

3. React Components, Props, State, Hooks & Routing

Q4: How is authentication state managed across LearnHub without Redux?

■ **Relevant File:** `client/src/context/AuthContext.jsx`

Answer: Global authentication state is managed using React's Context API inside `AuthContext.jsx`. The `AuthProvider` maintains user, token, and loading states. On initial mount, `useEffect` reads the JWT from `localStorage` and sends a GET request to `/api/auth/me`. If valid, the user state is populated across all child components via `useContext(AuthContext)`.

Interviewer Follow-up Questions:

- Follow-up 1: What happens if the user manually deletes the JWT token from `localStorage`?
- Follow-up 2: Why call `/api/auth/me` on page refresh instead of just decoding `localStorage` string?

Deep Dive Answer: If the token is deleted, `AuthContext` sets `token` and `user` to null, causing `ProtectedRoute` components to instantly redirect the user to `/login`. Calling `/api/auth/me` validates that the token hasn't been revoked or tampered with on the server.

Q5: How does client-side route protection work for Admin vs Student routes?

■ **Relevant File:** `client/src/components/ProtectedRoute.jsx` & `App.jsx`

Answer: `ProtectedRoute.jsx` acts as a route guard wrapper. It checks if `loading` is `true` (rendering a loading message), then verifies if user exists. If unauthenticated, it returns `.` If a `roleRequired` prop (such as `'admin'`) is provided and `user.role` does not match, it redirects to home `'/'`. If authorized, it renders `.`

Interviewer Follow-up Questions:

- Follow-up 1: Why use `inside ProtectedRoute`?
- Follow-up 2: Can a client bypass client-side `ProtectedRoute` by editing React state in dev tools?

Deep Dive Answer: allows nested route definitions in `App.jsx` under a single guard component. Editing client state in dev tools might reveal UI components visually, but any API call will still fail with `HTTP 401/403` because backend Express routes independently enforce auth middleware.

Q6: How does `LessonView.jsx` dynamically render lesson content (text vs embedded video)?

■ **Relevant File:** `client/src/pages/LessonView.jsx` (`renderContent`)

Answer: In `LessonView.jsx`, `renderContent` checks if the lesson content string starts with `'http://'` or `'https://'`. If it detects YouTube URLs (e.g. `youtube.com/watch?v=` or `youtu.be/`), it extracts the video ID and formats an embed link (`https://www.youtube.com/embed/{id}`) rendered inside a responsive `.` Otherwise, it renders formatted text inside a pre-wrapped `.`

Interviewer Follow-up Questions:

- Follow-up 1: What CSS technique ensures the YouTube iframe maintains a 16:9 aspect ratio?
- Follow-up 2: How does clicking a lesson in the sidebar change the displayed content?

Deep Dive Answer: The iframe is wrapped in a container with `position: relative`, `padding-bottom: 56.25%` (9/16 ratio), `height: 0`, and the iframe is set to `position: absolute` with `width: 100%` and `height: 100%`. Clicking a sidebar lesson updates the `activeLesson` state hook.

4. Node.js & Express APIs

Q7: How are REST API routes organized in LearnHub backend?

■ **Relevant File:** `server/server.js` & `server/routes/`

Answer: `server.js` acts as the entry point, mounting three modular router files: `/api/auth` (`authRoutes.js`), `/api/courses` (`courseRoutes.js`), and `/api/enrollments` (`enrollmentRoutes.js`). Middleware like `cors()` and `express.json()` are configured at the top level before route handlers.

Interviewer Follow-up Questions:

- Follow-up 1: Why split endpoints into separate routes and controllers folders?
- Follow-up 2: What does `express.json()` do under the hood?

Deep Dive Answer: Splitting routes and controllers enforces separation of concerns: route files declare URL paths and middleware stacks, while controller files contain pure business and database logic. `express.json()` parses incoming JSON request bodies and populates `req.body`.

5. Database Design & Mongoose

Q8: Detail the Mongoose data models and relationships built in LearnHub.

■ **Relevant File:** `server/models/ (User.js, Course.js, Lesson.js, Enrollment.js)`

Answer: User has name, email, password, role ('admin'/'student'). Course has title, description, instructorName, price, thumbnail, and lessons (array of ObjectId ref 'Lesson'). Lesson has courseId (ref 'Course'), title, content, orderNumber. Enrollment has studentId (ref 'User'), courseId (ref 'Course'), enrolledAt, and completedLessonIds (array of ObjectId ref 'Lesson').

Interviewer Follow-up Questions:

- Follow-up 1: Why put a compound unique index on Enrollment ({ studentId: 1, courseId: 1 })?
- Follow-up 2: Why reference lessons in Course instead of embedding lesson objects directly?

Deep Dive Answer: The compound unique index prevents duplicate enrollment records at the database level if a student clicks enroll twice concurrently. Referencing Lesson documents keeps Course documents lightweight and allows independent lesson ordering and updates.

Q9: How do Mongoose populate operations work in courseController.js and enrollmentController.js?

■ **Relevant File:** `server/controllers/courseController.js & enrollmentController.js`

Answer: Populate performs join-like references. In `getCourseById`, `Course.findById(id).populate({ path: 'lessons', options: { sort: { orderNumber: 1 } } })` fetches the course and populates its referenced lesson objects sorted by orderNumber. In `getMyEnrollments`, a nested populate (`path: 'courseId', populate: { path: 'lessons' }`) retrieves full course and lesson structures in a single query.

Interviewer Follow-up Questions:

- Follow-up 1: What is the performance cost of deep nested populate calls?
- Follow-up 2: How would you optimize populate queries for thousands of records?

Deep Dive Answer: Deep nested populates issue multiple underlying MongoDB queries. To optimize for high volume, select only necessary fields (e.g. `populate('studentId', 'name email')`) or use MongoDB aggregation pipelines (`$lookup`).

6. Authentication and Authorization

Q10: Walk me through the secret admin signup and JWT generation logic.

■ **Relevant File:** `server/controllers/authController.js (registerUser)`

Answer: When `registerUser` runs, it extracts name, email, password, adminCode from `req.body`. It verifies email uniqueness, then checks if `adminCode === process.env.ADMIN_SECRET_CODE`. If true, role is 'admin', else 'student'. It hashes the password using `bcrypt.hash(password, 10)`, saves the User document, and signs a JWT containing { id: user._id, role: user.role } with 30-day expiration.

Interviewer Follow-up Questions:

- Follow-up 1: How does protect middleware authenticate incoming HTTP requests?
- Follow-up 2: How does adminOnly middleware restrict admin routes?

Deep Dive Answer: `protect` reads the Authorization header ('Bearer '), verifies it with `jwt.verify` using `process.env.JWT_SECRET`, fetches User omitting password, and attaches it to `req.user`. `adminOnly` simply verifies `req.user && req.user.role === 'admin'`.

7. API Request / Response Flow

Q11: Trace the lifecycle of marking a lesson complete from button click to progress bar update.

■ **Relevant File:** [client/src/pages/LessonView.jsx](#) & [server/controllers/enrollmentController.js](#)

Answer: 1. Student checks 'Mark Completed' checkbox in LessonView.jsx. 2. handleToggleComplete sends POST to /api/enrollments/complete-lesson with header Bearer token and body { courseId, lessonId }. 3. Backend protect middleware decodes token and sets req.user. 4. markLessonComplete controller finds Enrollment by studentId and courseId. 5. It toggles lessonId inside completedLessonIds array using indexOf and splice/push. 6. Saves enrollment, calculates completedCount / totalLessons * 100 percentage. 7. Returns updated JSON payload. 8. React state hooks (completedLessonIds, progressPercentage) update, instantly re-rendering ProgressBar.

Interviewer Follow-up Questions:

- Follow-up 1: What response status is returned if enrollment is not found?
- Follow-up 2: How does the toggle feature allow unmarking a lesson?

Deep Dive Answer: HTTP 404 is returned if enrollment record doesn't exist. The toggle logic checks indexOf(lessonId); if found (> -1), it splices it out; if not found, it pushes it.

8. CRUD Operations

Q12: How does deleteCourse in courseController.js handle cascading deletions?

■ **Relevant File:** [server/controllers/courseController.js](#) (deleteCourse)

Answer: When an admin deletes a course, deleteCourse first verifies the course exists. It then executes Lesson.deleteMany({ courseId: course._id }) and Enrollment.deleteMany({ courseId: course._id }) before calling course.deleteOne(). This guarantees orphan lessons and enrollments are wiped clean.

Interviewer Follow-up Questions:

- Follow-up 1: What would happen if course.deleteOne() ran before deleting lessons?
- Follow-up 2: How could database transactions (session.startTransaction) improve safety here?

Deep Dive Answer: If course was deleted first and lesson deletion failed midway, database orphan records would remain. Using Mongoose sessions and transactions ensures all three deletions succeed or roll back atomically.

Q13: How does addLessonToCourse assign lesson order numbers?

■ **Relevant File:** [server/controllers/courseController.js](#) (addLessonToCourse)

Answer: addLessonToCourse checks if an orderNumber was provided in req.body. If provided, it converts it to Number(orderNumber). If omitted, it automatically calculates orderNumber = course.lessons.length + 1. It creates the Lesson document, pushes lesson._id into course.lessons array, and saves the course.

Interviewer Follow-up Questions:

- Follow-up 1: What happens if two lessons are assigned the exact same order number?
- Follow-up 2: How does getCourseById sort lessons when returning course details?

Deep Dive Answer: If duplicate order numbers exist, ties are resolved by Mongoose creation date. getCourseById explicitly applies populate options { sort: { orderNumber: 1 } } to sort ascending.

9. Error Handling & Validation

Q14: How is error handling structured across Express backend controllers?

■ **Relevant File:** `server/controllers/` (all controllers)

Answer: Every controller function wraps its execution block in try-catch. Input validation checks presence of required fields early (returning status 400 with a clear error message). Uncaught server errors or Mongoose schema failures trigger the catch block, returning `res.status(500).json({ message: error.message })`.

Interviewer Follow-up Questions:

- Follow-up 1: Why is returning generic 500 stack traces risky in production?
- Follow-up 2: How could a centralized Express error handler middleware replace repetitive try-catch blocks?

Deep Dive Answer: Exposing raw error stack traces can reveal database structure or internal file paths to attackers. A centralized error handling middleware (using `express-async-errors` or `next(err)`) sanitizes production errors while logging details internally.

10. Project Architecture

Q15: Describe the architectural layers of LearnHub from database to UI.

■ **Relevant File:** `Full Codebase Architecture`

Answer: LearnHub uses a clean 3-tier architecture: - Presentation Layer: React frontend (Vite) rendering JSX pages and components, managing state via `AuthContext` and `Hooks`. - Application Layer: Express REST API server running Node.js, utilizing middleware (CORS, JSON parser, JWT protect, `adminOnly`) and modular controllers for business logic. - Data Layer: MongoDB database mapped via Mongoose schemas (User, Course, Lesson, Enrollment).

Interviewer Follow-up Questions:

- Follow-up 1: How does Vite proxy resolve API calls during local development?
- Follow-up 2: How are client API calls decoupled from hardcoded port numbers?

Deep Dive Answer: `vite.config.js` configures a server proxy rule mapping `'/api'` to `'http://localhost:5000'`. Client fetch calls simply target `'/api/...'` without hardcoding backend ports.

11. Security Best Practices

Q16: What security measures were built into LearnHub?

■ **Relevant File:** `server/middleware/authMiddleware.js` & `server/controllers/authController.js`

Answer: 1. Passwords salted and hashed with `bcryptjs` prior to storage. 2. `ADMIN_SECRET_CODE` kept strictly server-side in `.env`. 3. Role authorization enforced server-side from decoded JWT claims. 4. Route middleware strips password hashes (`select('-password')`) when returning user profiles. 5. Mongoose schema sanitization protects against NoSQL injection.

Interviewer Follow-up Questions:

- Follow-up 1: What is XSS and how does React protect against it?
- Follow-up 2: Why store JWT in `localStorage` vs `httpOnly` cookies?

Deep Dive Answer: React automatically escapes JSX variable bindings, preventing Cross-Site Scripting (XSS). `LocalStorage` was chosen for simple SPA client-side token retrieval, though `httpOnly` cookies offer higher protection against XSS token theft.

12. Performance & Scalability

Q17: How would you scale LearnHub to support 100,000 active students?

■ **Relevant File:** [System Architecture & Database](#)

Answer: 1. Database Indexing: Ensure compound index on Enrollment (`{ studentId: 1, courseId: 1 }`) and single index on Lesson `courseId`. 2. Caching: Use Redis to cache public course catalog endpoints (`/api/courses`) to eliminate repetitive DB queries. 3. Media CDN: Serve lesson thumbnail images and video assets via AWS S3 / CloudFront CDN. 4. Server Statelessness: Express JWT auth is stateless, enabling horizontal scaling across multiple Node instances behind a Load Balancer.

Interviewer Follow-up Questions:

- Follow-up 1: How does Redis caching invalidate when an admin updates a course?
- Follow-up 2: What is database connection pooling in Mongoose?

Deep Dive Answer: When `createCourse` or `updateCourse` runs, the controller publishes a Redis key invalidation event. Mongoose automatically manages a pool of reusable MongoDB socket connections.

13. Deployment

Q18: How would you deploy LearnHub to production?

■ **Relevant File:** [README.md & Environment Setup](#)

Answer: Frontend: Build static assets via `npm run build` in Vite and host on Vercel or Netlify CDN. Backend: Deploy Express server on Render, Railway, or AWS EC2. Database: Provision a managed MongoDB Atlas cluster. Environment: Configure `MONGO_URI`, `JWT_SECRET`, `PORT`, and `ADMIN_SECRET_CODE` in cloud environment variables.

Interviewer Follow-up Questions:

- Follow-up 1: How do you configure CORS in production when frontend and backend are on different domains?
- Follow-up 2: What environment variable change is required for production Express server?

Deep Dive Answer: Pass origin options to `cors({ origin: 'https://learnhub.vercel.app' })`. Set `NODE_ENV=production` in Express server environment.

14. Code-Level Deep Audit Questions

Q19: In `courseController.js`, examine `getCourseStudents`. How does it compile student progress lists?

■ **Relevant File:** [server/controllers/courseController.js \(getCourseStudents\)](#)

Answer: `getCourseStudents` retrieves the course by ID with populated lessons. It then queries `Enrollment.find({ courseId: req.params.id }).populate('studentId', 'name email').populate('completedLessonIds')`. It maps over enrollments, computing `completedCount = e.completedLessonIds.length`, `totalLessons = course.lessons.length`, and `progressPercentage = Math.round((completedCount / totalLessons) * 100)` (defaulting to 0 if `totalLessons` is 0).

Interviewer Follow-up Questions:

- Follow-up 1: What happens if a student account was deleted but enrollment remains?
- Follow-up 2: Why populate only 'name email' for `studentId`?

Deep Dive Answer: If student was deleted, `e.studentId` is null; the map function handles this cleanly. Selecting only 'name email' minimizes network bandwidth and avoids sending password hashes.

Q20: In EditCourse.jsx, how are course details editing and lesson creation managed simultaneously?

■ **Relevant File:** `client/src/pages/EditCourse.jsx`

Answer: EditCourse.jsx maintains separate state hooks for course metadata (title, description, price, thumbnail, instructorName) and new lesson creation (lessonTitle, lessonContent, lessonOrder). It renders a two-column grid containing two independent forms: Form 1 triggers `handleUpdateCourse` (PUT `/api/courses/:id`), and Form 2 triggers `handleAddLesson` (POST `/api/courses/:id/lessons`).

Interviewer Follow-up Questions:

- Follow-up 1: How does EditCourse update the displayed lessons list after adding a new lesson?
- Follow-up 2: What happens if an admin submits a lesson with blank title or content?

Deep Dive Answer: After a successful lesson addition, `handleAddLesson` calls `fetchCourse()` to refresh the lessons state array. Blank inputs are blocked by HTML5 required attribute and backend 400 validation.

Q21: In CourseCard.jsx, how does the CTA button dynamically adapt to student vs admin views?

■ **Relevant File:** `client/src/components/CourseCard.jsx`

Answer: CourseCard receives props: `course`, `isEnrolled`, `progressPercentage`, `adminView`. If `adminView` is true, it renders 'Manage Course' linking to `/admin/courses/:id/edit`. If `isEnrolled` is true, it renders 'Continue Learning' linking to `/student/courses/:id/learn`. Otherwise, it renders 'View Details' linking to `/courses/:id`.

Interviewer Follow-up Questions:

- Follow-up 1: How does CourseCard handle broken image thumbnail URLs?
- Follow-up 2: Where is ProgressBar rendered inside CourseCard?

Deep Dive Answer: An `onError` handler replaces broken thumbnail sources with a placeholder fallback image. ProgressBar is rendered conditionally above the CTA button if `isEnrolled` is true.

Q22: In enrollmentController.js, examine getMyEnrollments. How does it handle nested population?

■ **Relevant File:** `server/controllers/enrollmentController.js (getMyEnrollments)`

Answer: `getMyEnrollments` queries `Enrollment.find({ studentId: req.user._id }).populate({ path: 'courseId', populate: { path: 'lessons' } })`. It transforms the enrollment array, calculating completion count and progress percentage for each course, returning an array of enrolled course objects.

Interviewer Follow-up Questions:

- Follow-up 1: Why use `.filter(Boolean)` at the end of the map array transformation?
- Follow-up 2: What happens if an enrolled course was deleted by an admin?

Deep Dive Answer: If an admin deleted a course, `e.courseId` returns null; `.filter(Boolean)` strips out null objects so the student dashboard doesn't crash.

15. Possible Bugs & Debugging Scenarios

Q23: What bug occurs if a student attempts to enroll in a course twice, and how is it prevented?

■ **Relevant File:** `server/controllers/enrollmentController.js (enrollInCourse)`

Answer: If a student double-clicks 'Enroll', two concurrent requests might reach the server. `enrollInCourse` first checks `Enrollment.findOne({ studentId, courseId })`. If found, it returns status 400 'Already enrolled'. Additionally, the Mongoose schema defines a compound index `enrollmentSchema.index({ studentId: 1, courseId: 1 }, { unique: true })`, guaranteeing MongoDB rejects duplicate insertions.

Interviewer Follow-up Questions:

- Follow-up 1: How does the client UI handle the 400 error message?
- Follow-up 2: What is a race condition in database operations?

Deep Dive Answer: The client catches the error and displays a user-friendly alert box. A race condition occurs when two requests arrive simultaneously before the first query completes; the unique index acts as a strict guard.

Q24: What happens if a user's JWT expires while navigating lessons, and how is it handled?

■ **Relevant File:** `client/src/context/AuthContext.jsx & server/middleware/authMiddleware.js`

Answer: When the JWT expires, backend protect middleware returns HTTP 401 'Not authorized, token invalid'. Client fetch calls in pages catch non-OK responses. `AuthContext`'s initial `/api/auth/me` fetch clears `localStorage` and resets user to null, sending the user to `/login`.

Interviewer Follow-up Questions:

- Follow-up 1: How can an HTTP interceptor streamline 401 response handling globally?
- Follow-up 2: What is the benefit of setting JWT expiration to 30 days vs 1 hour?

Deep Dive Answer: A global fetch wrapper or Axios interceptor catches 401 status codes across all API calls and triggers logout automatically. 30 days provides seamless user convenience, while 1 hour enhances security.

16. Future Improvements

Q25: What key features would you add to version 2.0 of LearnHub?

■ **Relevant File:** `Future Roadmap`

Answer: 1. Real-time Payment Integration: Integrate Stripe / Razorpay for paid course checkout. 2. Quiz & Certificate Module: Add multiple-choice quizzes at the end of courses with auto-generated PDF completion certificates. 3. Discussion Forums: Enable student-instructor Q&A; threads under each lesson using Socket.io. 4. Video Uploads: Direct video file uploads to AWS S3 storage with HLS adaptive streaming.

Interviewer Follow-up Questions:

- Follow-up 1: How would certificate generation be implemented?
- Follow-up 2: How would Socket.io enable live discussion comments?

Deep Dive Answer: When progress reaches 100%, trigger a PDF generation service (like ReportLab or PDFKit) to issue a signed certificate. Socket.io websockets broadcast real-time discussion comments.

Part 2: 10 Core Data Structures & Algorithms (DSA) Questions

Hand-picked DSA problems mapped to LearnHub features, complete with Java solutions, approaches, and complexity analysis.

DSA 1: Array: Find Two Courses matching Student Budget (Two Sum Variant)

Concept: [Array](#) / [HashMap](#)

Problem Statement: Given an array of course prices and a target student budget, find the indices of two distinct courses whose prices sum exactly to target budget.

Test Cases:

Input: prices = [29, 49, 99, 149], budget = 148

Output: [1, 2] (49 + 99 = 148)

Approach: Use a HashMap to store price values and their corresponding array indices. For each course price, calculate complement = budget - price. If complement exists in map, return indices.

Java Solution Code:

```
import java.util.HashMap;
import java.util.Map;

public class CourseBudgetFinder {
    public static int[] findTwoCourses(int[] prices, int targetBudget) {
        Map map = new HashMap<>();
        for (int i = 0; i < prices.length; i++) {
            int complement = targetBudget - prices[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(prices[i], i);
        }
        return new int[] {};
    }
}
```

Complexity: Time Complexity: $O(N)$ single pass | Space Complexity: $O(N)$ for HashMap storage

DSA 2: String: Clean and Validate Course Title Slug (String Formatting)

Concept: [String / Regex](#)

Problem Statement: Convert a course title string into a sanitized URL slug by lowercasing, stripping special characters, replacing spaces with hyphens, and removing duplicate hyphens.

Test Cases:

Input: title = " Learn MERN Stack: 2026 Edition!! "

Output: "learn-mern-stack-2026-edition"

Approach: Trim whitespace, convert to lowercase, replace non-alphanumeric characters with spaces, split by whitespace, and join non-empty tokens with single hyphens.

Java Solution Code:

```
public class CourseSlugGenerator {  
    public static String generateSlug(String title) {  
        if (title == null || title.trim().isEmpty()) return "";  
        String cleaned = title.trim().toLowerCase().replaceAll("[^a-z0-9\\s]", "");  
        String[] words = cleaned.split("\\s+");  
        return String.join("-", words);  
    }  
}
```

Complexity: *Time Complexity: $O(N)$ length of string | Space Complexity: $O(N)$ string building*

DSA 3: HashMap: Group Students by Progress Tier (Frequency & Categorization)

Concept: [HashMap / Grouping](#)

Problem Statement: Given an array of student completion percentages, categorize students into three tiers: 'Beginner' (< 30%), 'Intermediate' (30% - 70%), and 'Advanced' (> 70%), returning student counts per tier.

Test Cases:

Input: progress = [15, 45, 80, 95, 25, 60]

Output: {Beginner=2, Intermediate=2, Advanced=2}

Approach: Iterate through completion percentages, evaluate conditional boundaries, and increment counter map values accordingly.

Java Solution Code:

```
import java.util.HashMap;
import java.util.Map;

public class StudentProgressCategorizer {
    public static Map categorizeStudents(int[] progress) {
        Map map = new HashMap<>();
        map.put("Beginner", 0);
        map.put("Intermediate", 0);
        map.put("Advanced", 0);

        for (int p : progress) {
            if (p < 30) {
                map.put("Beginner", map.get("Beginner") + 1);
            } else if (p <= 70) {
                map.put("Intermediate", map.get("Intermediate") + 1);
            } else {
                map.put("Advanced", map.get("Advanced") + 1);
            }
        }
        return map;
    }
}
```

Complexity: *Time Complexity: $O(N)$ array iteration | Space Complexity: $O(1)$ fixed 3 keys*

DSA 4: Sliding Window: Maximum Average Lesson Duration in Window K

Concept: Sliding Window

Problem Statement: Given an array of lesson duration minutes and contiguous window size K, find the maximum average duration of any K contiguous lessons.

Test Cases:

Input: durations = [10, 20, 30, 40, 15, 25], K = 3

Output: 30.0 (lessons [20, 30, 40] avg = 30.0)

Approach: Compute sum of first K elements. Slide window from index K to end, adding new right element and subtracting left element, tracking maximum average.

Java Solution Code:

```
public class MaxLessonDurationWindow {
    public static double maxAverageDuration(int[] durations, int k) {
        if (durations.length < k || k <= 0) return 0.0;
        double currentSum = 0;
        for (int i = 0; i < k; i++) {
            currentSum += durations[i];
        }
        double maxSum = currentSum;
        for (int i = k; i < durations.length; i++) {
            currentSum += durations[i] - durations[i - k];
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum / k;
    }
}
```

Complexity: Time Complexity: $O(N)$ linear sliding window | Space Complexity: $O(1)$ auxiliary

DSA 5: Binary Search: Search Lesson Order Number in Sorted Syllabus

Concept: [Binary Search](#)

Problem Statement: Given an array of sorted lesson order numbers, find index of target lesson order number using Binary Search.

Test Cases:

Input: orders = [1, 2, 4, 7, 9, 12, 15], target = 9

Output: Index 4

Approach: Maintain left and right pointers. Compute mid pointer. If target matches mid, return mid. If target > mid, search right half; else left half.

Java Solution Code:

```
public class SyllabusBinarySearch {  
    public static int findLessonOrder(int[] orders, int target) {  
        int left = 0, right = orders.length - 1;  
        while (left <= right) {  
            int mid = left + (right - left) / 2;  
            if (orders[mid] == target) {  
                return mid;  
            } else if (orders[mid] < target) {  
                left = mid + 1;  
            } else {  
                right = mid - 1;  
            }  
        }  
        return -1;  
    }  
}
```

Complexity: Time Complexity: $O(\log N)$ logarithmic search | Space Complexity: $O(1)$ iterative

DSA 6: DFS / BFS: Course Prerequisite Dependency Traversal (Topological Sort)

Concept: [Graph / Topological Sort / Kahn's Algorithm](#)

Problem Statement: Determine if all N courses can be completed given prerequisite pairs [course, prerequisite]. Return valid course learning sequence.

Test Cases:

Input: numCourses = 4, prerequisites = [[1,0], [2,1], [3,2]]

Output: [0, 1, 2, 3]

Approach: Build adjacency list and in-degree array. Use BFS Queue (Kahn's Algorithm) starting with 0 in-degree nodes. Process dependencies sequentially.

Java Solution Code:

```
import java.util.*;

public class CoursePrerequisiteOrder {
    public static int[] findOrder(int numCourses, int[][] prerequisites) {
        List<Integer> adj = new ArrayList<>();
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());

        for (int[] p : prerequisites) {
            adj.get(p[1]).add(p[0]);
            inDegree[p[0]]++;
        }

        Queue<Integer> q = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) q.add(i);
        }

        int[] result = new int[numCourses];
        int index = 0;
        while (!q.isEmpty()) {
            int curr = q.poll();
            result[index++] = curr;
            for (int next : adj.get(curr)) {
                if (--inDegree[next] == 0) q.add(next);
            }
        }
        return index == numCourses ? result : new int[0];
    }
}
```

Complexity: Time Complexity: $O(V + E)$ vertices & edges | Space Complexity: $O(V + E)$ graph structure

DSA 7: Dynamic Programming: 0/1 Knapsack Course Selection for Max Skill Gain

Concept: Dynamic Programming / 0-1 Knapsack

Problem Statement: Given course study hours (weights), skill gain points (values), and maximum available study hours (capacity), select optimal courses to maximize skill points.

Test Cases:

Input: hours = [2, 3, 4, 5], skills = [3, 4, 5, 6], maxHours = 5

Output: 7 (courses 1 & 2: hours 2+3=5, skills 3+4=7)

Approach: Use 2D DP table $dp[i][w]$ representing max skill gain using first i courses with w available hours.

Java Solution Code:

```
public class CourseKnapsackSelector {
    public static int maxSkillGain(int[] hours, int[] skills, int maxHours) {
        int n = hours.length;
        int[][] dp = new int[n + 1][maxHours + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 1; w <= maxHours; w++) {
                if (hours[i - 1] <= w) {
                    dp[i][w] = Math.max(skills[i - 1] + dp[i - 1][w - hours[i - 1]], dp[i - 1][w]);
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }
        return dp[n][maxHours];
    }
}
```

Complexity: Time Complexity: $O(N * \text{maxHours})$ | Space Complexity: $O(N * \text{maxHours})$ DP table

DSA 8: Stack: Validate Nested Syllabus Markup Tags (Valid Parentheses)

Concept: [Stack / String Parsing](#)

Problem Statement: Validate whether nested syllabus formatting tags (e.g. '[', ']', '{', '}', '(', ')') are properly closed in correct sequence.

Test Cases:

Input: tags = "{[()]}"

Output: true

Input: tags = "{[(])}"

Output: false

Approach: Use a Stack. Push expected closing brackets when an opening bracket is encountered. If closing bracket is encountered, pop and match with stack top.

Java Solution Code:

```
import java.util.Stack;

public class SyllabusTagValidator {
    public static boolean isValidTagSequence(String s) {
        Stack stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(') stack.push(')');
            else if (c == '{') stack.push('}');
            else if (c == '[') stack.push(']');
            else if (stack.isEmpty() || stack.pop() != c) return false;
        }
        return stack.isEmpty();
    }
}
```

Complexity: *Time Complexity: $O(N)$ string traversal | Space Complexity: $O(N)$ stack memory*

DSA 9: Sliding Window & Map: Longest Substring of Unique Lesson Titles

Concept: [Sliding Window / HashMap](#)

Problem Statement: Given a sequence of lesson title keywords, find the length of the longest contiguous sub-sequence containing no duplicate keywords.

Test Cases:

Input: keywords = ["react", "hooks", "state", "react", "router"]

Output: 3 (["hooks", "state", "react"])

Approach: Maintain sliding window [left, right] and a Map storing last seen index of each keyword. Update left pointer to $\max(\text{left}, \text{lastSeen} + 1)$.

Java Solution Code:

```
import java.util.HashMap;
import java.util.Map;

public class UniqueLessonSequence {
    public static int longestUniqueLessons(String[] keywords) {
        Map map = new HashMap<>();
        int maxLength = 0, left = 0;
        for (int right = 0; right < keywords.length; right++) {
            if (map.containsKey(keywords[right])) {
                left = Math.max(left, map.get(keywords[right]) + 1);
            }
            map.put(keywords[right], right);
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

Complexity: *Time Complexity: $O(N)$ single pass | Space Complexity: $O(N)$ hash map storage*

DSA 10: Tree DFS: Maximum Depth of Course Category Tree

Concept: [Tree / Depth First Search \(DFS\)](#)

Problem Statement: Calculate the maximum depth (height) of a hierarchical course category tree node structure.

Test Cases:

Input: Root Category 'Web Dev' -> 'Frontend' -> 'React' -> 'Hooks'

Output: Depth 4

Approach: Recursive Depth First Search (DFS). Base case: if node is null return 0. Recursively calculate max depth of sub-categories + 1.

Java Solution Code:

```
import java.util.List;
import java.util.ArrayList;

class CategoryNode {
    String name;
    List children;

    public CategoryNode(String name) {
        this.name = name;
        this.children = new ArrayList<>();
    }
}

public class CourseTreeDepthCalculator {
    public static int maxDepth(CategoryNode root) {
        if (root == null) return 0;
        int maxChildDepth = 0;
        for (CategoryNode child : root.children) {
            maxChildDepth = Math.max(maxChildDepth, maxDepth(child));
        }
        return 1 + maxChildDepth;
    }
}
```

Complexity: *Time Complexity: $O(V)$ nodes in category tree | Space Complexity: $O(H)$ recursion stack height*